A Distributed Multimodal Retrieval System Using CLIP Embeddings

Paige X. Phillips
CSCI 5253 - Datacenter Scale Computing
April 24, 2026

1 Title & Participants

Title: A Distributed Multimodal Retrieval System Using CLIP Embeddings
Participant: Paige Phillips (Individual Project)

2 Project Goals

The goal of this project was to design and implement a distributed multimodal retrieval system that connects images and text through a shared embedding space. The system allows a user to upload an image through a REST API and receive the most relevant text labels, such as "forest," "mountain," or "coastline." In addition, the system supports text-to-image retrieval, where a user can submit a text query and retrieve the most relevant images based on semantic similarity.

The primary objective of this project was not to train a new machine learning model, but rather to design a system that demonstrates how multimodal AI workflows can be supported using modern software components. The focus was on building a modular system architecture that integrates an API layer, embedding generation, vector similarity search, and caching, while simulating the behavior of a distributed pipeline. While the original proposal outlined a fully cloud-based distributed system, the final implementation prioritizes a working local system that captures the core behavior of a distributed pipeline.

System input and output are centered around two main workflows. For image-to-text retrieval, a user uploads an image, and the system generates an embedding using a pretrained CLIP model and compares it to predefined text embeddings to return the most relevant labels. For text-to-image retrieval, a user submits a text query, which is converted into an embedding and compared against stored image embeddings to retrieve the most similar images. In both

cases, image and text inputs are represented in the same embedding space, allowing similarity search to be performed using cosine similarity.

Overall, the goal of this project was to demonstrate how a multimodal retrieval system can be built using existing machine learning models and integrated into a structured system design. The project emphasizes system functionality, modularity, and real-world applicability rather than model training or optimization.

3 Software & Hardware Components

The final system is composed of several software components that work together to support multimodal retrieval. While the original proposal outlined a fully cloud-based distributed architecture, the implemented system focuses on a modular local deployment that demonstrates the core functionality of a distributed pipeline.

The API layer is implemented using FastAPI, which handles incoming user requests, including image uploads and text queries. FastAPI serves as the main entry point to the system and manages routing, request validation, and response formatting. It also provides an interactive interface through Swagger, which was used during development for testing and debugging API endpoints.

Embedding generation is performed using PyTorch with a pretrained CLIP model. This component is responsible for converting both image and text inputs into embeddings within a shared vector space. This shared embedding space enables the system to perform cross-modal retrieval, allowing meaningful comparisons between images and text.

For similarity search, the system uses FAISS, which efficiently computes nearest neighbor searches over high-dimensional embeddings. FAISS serves as the core retrieval engine of the system, allowing the system to quickly identify the most relevant matches based on cosine similarity.

To improve performance, a caching layer is implemented using Redis. Redis stores the results of previously processed queries so that repeated requests can be returned immediately without recomputing embeddings or performing additional similarity searches. This reduces latency and improves overall system efficiency.

The system is containerized using Docker, which ensures consistent execution across environments and allows different components of the system to be run in isolated containers.

While the system is not fully deployed across multiple machines, containerization helps simulate aspects of a distributed architecture.

From a hardware perspective, the system was developed and tested on a local machine using CPU-based inference. Although this introduces some latency during embedding generation, it was sufficient for demonstrating the functionality of the system and the interaction between components.

4 Architectural Diagram

The system follows a modular pipeline architecture in which each component is responsible for a specific stage of processing. Although the system is implemented locally, it is designed to simulate the behavior of a distributed system by separating functionality into distinct stages, including request handling, embedding generation, similarity search, and caching.

The process begins when a user submits either an image or a text query through the REST API. The API, built using FastAPI, acts as the entry point to the system and is responsible for handling incoming requests, validating inputs, and routing data through the pipeline.

For image inputs, the API performs basic preprocessing and passes the image to the embedding generation stage. For text inputs, the API directly converts the query into an embedding. In both cases, embeddings are generated using a pretrained CLIP model implemented in PyTorch. This ensures that both image and text data are represented within the same shared vector space.

Once embeddings are generated, they are passed to the similarity search component, which is implemented using FAISS. FAISS performs nearest neighbor search using cosine similarity to identify the most relevant matches between query embeddings and stored embeddings. The system retrieves the top matching results based on similarity scores.

To improve efficiency, the system incorporates a caching layer using Redis. Before performing embedding generation or similarity search, the system checks whether a query has been processed previously. If a cached result is found, it is returned immediately, reducing computation time and improving response latency. If no cached result exists, the system proceeds with the full retrieval pipeline and stores the result for future use.

All components are containerized using Docker, which allows the system to be executed in a consistent environment and supports a modular design. While the system does not operate

across multiple machines, the separation of responsibilities between components reflects the structure of a distributed system.

5 Component Interaction

The system operates as a modular pipeline where each component is responsible for a specific stage of processing. While the system is implemented locally, the interaction between components reflects the structure of a distributed system, with clearly defined responsibilities at each stage. The workflow proceeds as follows:

1. **User Request (User → API):** The process begins when a user submits either an image upload or a text query through the REST API. The API serves as the entry point to the system and handles all incoming requests.

2. **Request Processing (API → Processing Layer):** The API performs input validation and basic preprocessing. For image uploads, this includes saving or preparing the file for embedding generation. For text queries, the input is cleaned and prepared for encoding.

3. **Embedding Generation (Processing Layer → CLIP Model):** The system uses a pretrained CLIP model implemented in PyTorch to convert both image and text inputs into embeddings. These embeddings represent the semantic content of the input in a shared vector space.

4. **Similarity Search (Embedding → FAISS):** The generated embedding is passed to the similarity search component, which is implemented using FAISS. FAISS compares the query embedding against stored embeddings and retrieves the most similar results based on cosine similarity.

5. **Caching Check (Redis):** Before performing embedding generation and similarity search, the system checks whether the query has already been processed. This is done using Redis. If a cached result is found, the system bypasses the remaining steps and returns the stored output immediately.

6. **Result Formatting (FAISS → API):** The retrieved results, including similarity scores and associated labels or image references, are formatted into a structured response. This ensures consistency in how results are returned to the user.

7. **Response to User (API → User):** The API sends the final results back to the user in JSON format. For image-to-text queries, this includes the most relevant labels. For text-to-image queries, this includes the most relevant images or references.

6 Debugging & Testing

The system was tested and debugged incrementally to ensure that each component functioned correctly both individually and as part of the full pipeline. Because the project integrates multiple components, including the API, embedding generation, similarity search, and caching, a step-by-step testing approach was necessary to identify and resolve issues effectively.

Initial testing focused on validating individual components in isolation. The API endpoints were tested using the built-in interactive documentation provided by FastAPI, which allowed for rapid verification of image uploads and text query handling. This helped ensure that requests were being received correctly and that responses were properly formatted.

As components were integrated, debugging efforts focused on identifying issues in data flow between stages. One common challenge involved ensuring that embeddings generated from both image and text inputs were consistent in shape and compatible with the similarity search process. Logging was used to track embedding dimensions and intermediate outputs, which helped identify and resolve mismatches.

Additional debugging was required for file handling during image uploads, including ensuring that images were correctly saved, processed, and passed into the embedding generation stage. Errors related to file paths and input formatting were resolved through iterative testing and validation.

The caching layer was also tested to confirm that repeated queries were correctly identified and returned from memory. This involved submitting identical queries multiple times and verifying that cached results were used instead of recomputing embeddings and performing

similarity search. This testing confirmed that the caching mechanism reduced response time and improved efficiency.

Testing was primarily conducted through manual interaction with the API, including submitting various image and text queries and verifying that the returned results were reasonable and consistent. While large-scale automated or load testing was not implemented, the system was tested across multiple typical use cases to ensure reliability and correctness.

Overall, the debugging and testing process focused on validating component functionality, ensuring smooth integration between system stages, and confirming that the system produced accurate and consistent retrieval results.

7 Requirements Justification

This project satisfies the course requirement of integrating multiple datacenter-scale system components by combining several technologies that together support a multimodal retrieval pipeline. While the final implementation was completed in a local environment rather than a full cloud deployment, the system demonstrates how different components interact to simulate a distributed architecture.

The system includes several key components that each serve a distinct role. The REST API, implemented using FastAPI, acts as the entry point to the system and is responsible for handling user requests, routing data, and returning results. This component enables interaction between the user and the system.

Embedding generation is performed using PyTorch with a pretrained CLIP model. This allows both image and text inputs to be represented in a shared embedding space, which is necessary for performing cross-modal retrieval. The use of a pretrained model avoids the need for costly training while still providing meaningful semantic representations.

The vector similarity search component is implemented using FAISS. This component is responsible for efficiently retrieving the most relevant results by comparing high-dimensional embeddings. FAISS is well-suited for this task due to its speed and scalability for nearest neighbor search.

A caching layer is implemented using Redis to improve system performance. By storing the results of frequently repeated queries, Redis reduces redundant computation and allows the system to respond more quickly to common requests.

Finally, the system is containerized using Docker, which ensures consistent execution and supports modular system design. Containerization allows components to be isolated and managed more effectively, which is an important aspect of distributed system development.

Together, these components demonstrate how multiple technologies can be integrated to support a functional multimodal retrieval system. The project highlights the interaction between API services, machine learning models, vector databases, caching systems, and containerization, which aligns with the core objectives of datacenter-scale computing.

8 Scope & Relevance

This project was designed to be ambitious while still achievable within the scope of the semester. While the original proposal outlined a fully cloud-based distributed system, the final implementation focused on building a working local system that demonstrates the core components and behavior of a multimodal retrieval pipeline. This approach allowed the system to be developed, tested, and debugged in a controlled environment while still reflecting the structure of a distributed system.

The project is more complex than individual course assignments because it integrates multiple components into a single system, including an API, embedding generation using a pretrained model, vector similarity search, caching, and containerization. Each of these components was implemented and connected as part of a complete pipeline, requiring careful coordination and debugging to ensure that data flowed correctly between stages. This directly aligns with the learning objectives of datacenter-scale computing, particularly in understanding how independent services interact within a larger system.

A key challenge in this project was managing the integration between components rather than developing new machine learning models. Ensuring compatibility between embeddings, handling data flow between stages, and maintaining consistent system behavior required iterative testing and refinement. This reflects real-world system design, where building reliable infrastructure around machine learning models is often more complex than the models themselves.

This project is also relevant to real-world applications of multimodal AI systems. Image–text retrieval systems are used in search engines, recommendation systems, and environmental or

geographic analysis. By focusing on system design rather than model training, this project highlights the practical challenges of deploying AI systems in a structured and scalable way.

9 Limitations & Future Work

While the system successfully demonstrates a working multimodal retrieval pipeline, several limitations were identified during development. One of the primary limitations is that the system relies on CPU-based inference for embedding generation. This introduces latency, particularly for image processing, and would be significantly improved with GPU acceleration.

Another limitation is the scale of the dataset and retrieval system. The current implementation operates on a relatively small set of predefined labels and images, which limits the diversity and accuracy of retrieval results. As the dataset grows, additional optimizations would be required to maintain efficient similarity search performance, such as improved indexing strategies or more advanced vector database configurations.

The system is also implemented as a local, modular pipeline rather than a fully distributed cloud deployment. While the architecture is designed to reflect a distributed system, components such as message queues, cloud storage, and horizontal scaling were not fully implemented. As a result, the system has not been tested under high concurrency or large-scale workloads.

In terms of model performance, the system uses a pretrained CLIP model without fine-tuning. While this allows for general semantic understanding, it may not perform optimally for domain-specific tasks, such as highly specialized geographic or satellite image classification.

Future improvements to the system would focus on both performance and scalability. One direction would be to deploy the system in a cloud environment using container orchestration and distributed services to better support concurrent users and larger datasets. Another improvement would be to incorporate GPU-based inference to reduce latency during embedding generation. Expanding the dataset and introducing more diverse labels would also improve retrieval quality.

Additional enhancements could include implementing more robust evaluation metrics, such as Recall@K across larger datasets, and integrating user feedback mechanisms to refine retrieval results over time. These improvements would move the system closer to a production-level multimodal retrieval application.